

API_REFERENCE

LLM Verifier API Reference

OpenAPI Specification

```
openapi: 3.0.3
info:
  title: LLM Verifier API
  version: 2.0.0
  description: REST API for AI model verification and benchmarking
  contact:
    name: LLM Verifier Team
    url: https://github.com/your-org/llm-verifier
servers:
  - url: http://localhost:8080
    description: Local development server
  - url: https://api.llm-verifier.dev
    description: Production server

paths:
  /health:
    get:
      summary: Health check endpoint
      responses:
        '200':
          description: Service is healthy
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/HealthResponse'

  /api/v1/verify:
    post:
      summary: Verify a single model
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/VerificationRequest'
      responses:
```

```

    '200':
      description: Verification completed
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/VerificationResult'
    '400':
      description: Invalid request
    '500':
      description: Internal server error

/api/v1/verify/batch:
  post:
    summary: Verify multiple models
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/
BatchVerificationRequest'
    responses:
      '200':
        description: Batch verification completed
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/
BatchVerificationResult'

/api/v1/models:
  get:
    summary: List available models
    parameters:
      - name: provider
        in: query
        schema:
          type: string
        description: Filter by provider
      - name: min_score
        in: query
        schema:
          type: number
        description: Minimum score threshold
    responses:
      '200':
        description: List of models
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ModelList'

/api/v1/providers:

```

```

get:
  summary: List supported providers
  responses:
    '200':
      description: List of providers
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ProviderList'

/api/v1/export:
  post:
    summary: Export configuration
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ExportRequest'
    responses:
      '200':
        description: Configuration exported
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ExportResponse'

/api/v1/analytics:
  get:
    summary: Get usage analytics
    responses:
      '200':
        description: Analytics data
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AnalyticsResponse'

/api/v1/migrate:
  post:
    summary: Migrate configuration
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MigrationRequest'
    responses:
      '200':
        description: Migration completed
        content:
          application/json:

```

```
schema:
  $ref: '#/components/schemas/MigrationResponse'
```

Data Models

Core Data Structures

VerificationRequest

```
type VerificationRequest struct {
    ModelID      string          `json:"model_id" validate:"required"`
    Provider     string          `json:"provider" validate:"required"`
    APIKey       string          `json:"api_key,omitempty"`
    Endpoint     string          `json:"endpoint,omitempty"`
    TestCases    []TestCase      `json:"test_cases,omitempty"`
    Timeout      time.Duration   `json:"timeout,omitempty"`
    Options      map[string]interface{} `json:"options,omitempty"`
}
```

VerificationResult

```
type VerificationResult struct {
    ModelInfo      ModelInfo      `json:"model_info"`
    PerformanceScores PerformanceScore `json:"performance_scores"`
    TestResults    []TestResult   `json:"test_results,omitempty"`
    Error          string         `json:"error,omitempty"`
    Timestamp      time.Time      `json:"timestamp"`
    Duration       time.Duration  `json:"duration"`
}
```

PerformanceScore

```
type PerformanceScore struct {
    OverallScore    float64 `json:"overall_score" // 0-100`
    CodeCapability  float64 `json:"code_capability" // 0-100`
    Responsiveness  float64 `json:"responsiveness" // 0-100 (lower is better)`
    Reliability     float64 `json:"reliability" // 0-100`
    FeatureRichness float64 `json:"feature_richness" // 0-100`
    ValueProposition float64 `json:"value_proposition" // 0-100`
}
```

ModelInfo

```
type ModelInfo struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    Provider      string    `json:"provider"`
    Endpoint      string    `json:"endpoint"`
    Capabilities  []string  `json:"capabilities"`
    ContextWindow int       `json:"context_window"`
    MaxTokens     int       `json:"max_tokens"`
    InputPricing  float64   `json:"input_pricing" // per 1M tokens`
    OutputPricing float64   `json:"output_pricing" // per 1M tokens`
    SupportsStreaming bool    `json:"supports_streaming"`
    SupportsTools bool     `json:"supports_tools"`
    ReleaseDate   string    `json:"release_date"`
}
```

Configuration Structures

ExportRequest

```
type ExportRequest struct {
    Format      string    `json:"format" validate:"required,oneof=opencode crush claude-code"`
    Providers   []string  `json:"providers,omitempty"`
    MinScore    float64   `json:"min_score,omitempty"`
    IncludeAPIKeys bool      `json:"include_api_keys,omitempty"`
    OutputPath  string    `json:"output_path,omitempty"`
    Compression bool      `json:"compression,omitempty"`
}
```

OpenCodeConfig (Generated)

```
type OpenCodeConfig struct {
    Schema      string    `json:"$schema"`
    Data        map[string]interface{} `json:"data"`
    Providers   map[string]interface{} `json:"providers"`
    Agents      map[string]interface{} `json:"agents"`
    TUI         map[string]interface{} `json:"tui"`
    Shell       map[string]interface{} `json:"shell"`
    AutoCompact bool      `json:"autoCompact"`
    Debug       bool      `json:"debug"`
    DebugLSP    bool      `json:"debugLSP"`
}
```

Analytics Structures

AnalyticsResponse

```
type AnalyticsResponse struct {
    TotalExports      int
        `json:"total_exports"`
    SuccessfulExports int
        `json:"successful_exports"`
    FailedExports     int
        `json:"failed_exports"`
    SuccessRate       string
        `json:"success_rate"`
    UniqueProviders   int
        `json:"unique_providers"`
    UniqueModels      int
        `json:"unique_models"`
    LastExportTime     time.Time
        `json:"last_export_time"`
    PopularProviders   []AnalyticsItem
        `json:"popular_providers"`
    PopularModels      []AnalyticsItem
        `json:"popular_models"`
    AgentPreferences  []AnalyticsItem
        `json:"agent_preferences"`
    ExportHistory      []ExportHistoryEntry
        `json:"export_history"`
}

type AnalyticsItem struct {
    Name string `json:"name"`
    Count int    `json:"count"`
}
```

Error Codes

HTTP Status Codes

- **200 OK:** Request successful
- **400 Bad Request:** Invalid request parameters
- **401 Unauthorized:** Invalid API key
- **403 Forbidden:** Insufficient permissions
- **404 Not Found:** Resource not found
- **429 Too Many Requests:** Rate limit exceeded
- **500 Internal Server Error:** Server error
- **502 Bad Gateway:** Provider API error
- **503 Service Unavailable:** Service temporarily unavailable

Application Error Codes

```
const (
    // Provider errors
```

```

ErrProviderNotSupported = "PROVIDER_NOT_SUPPORTED"
ErrProviderAPIError     = "PROVIDER_API_ERROR"
ErrProviderTimeout      = "PROVIDER_TIMEOUT"
ErrProviderRateLimit    = "PROVIDER_RATE_LIMIT"

// Model errors
ErrModelNotFound        = "MODEL_NOT_FOUND"
ErrModelNotAvailable    = "MODEL_NOT_AVAILABLE"
ErrModelConfiguration   = "MODEL_CONFIGURATION_ERROR"

// Verification errors
ErrVerificationFailed    = "VERIFICATION_FAILED"
ErrVerificationTimeout   = "VERIFICATION_TIMEOUT"
ErrTestCaseFailed        = "TEST_CASE_FAILED"

// Configuration errors
ErrConfigInvalid         = "CONFIG_INVALID"
ErrConfigMigration       = "CONFIG_MIGRATION_ERROR"
ErrExportFailed          = "EXPORT_FAILED"

// System errors
ErrDatabaseError         = "DATABASE_ERROR"
ErrFileSystemService     = "FILE_SYSTEM_ERROR"
ErrNetworkError          = "NETWORK_ERROR"
)

```

Rate Limiting

Default Limits

- **Per Minute:** 60 requests
- **Per Hour:** 1000 requests
- **Per Day:** 10000 requests
- **Concurrent:** 10 simultaneous verifications

Custom Limits

Rate limits can be configured per provider:

```

{
  "rate_limits": {
    "openai": {
      "requests_per_minute": 50,
      "requests_per_hour": 500
    },
    "anthropic": {
      "requests_per_minute": 30,
      "requests_per_hour": 300
    }
  }
}

```

Authentication

API Key Authentication

All API requests require authentication:

```
curl -X POST "http://localhost:8080/api/v1/verify" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model_id": "gpt-4o",
    "provider": "openai",
    "api_key": "sk-your-key"
  }'
```

Provider API Keys

Provider API keys are required for verification:

```
{
  "openai_api_key": "sk-...",
  "anthropic_api_key": "sk-ant-...",
  "google_api_key": "...",
  "groq_api_key": "..."
}
```

WebSocket Streaming

Real-time Verification Updates

```
const ws = new WebSocket('ws://localhost:8080/ws/verify');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  console.log('Verification progress:', data);
};

// Start verification
ws.send(JSON.stringify({
  type: 'start_verification',
  model_id: 'gpt-4o',
  provider: 'openai'
}));
```


SDK Examples

Go SDK

```
package main

import (
    "context"
    "log"

    "github.com/your-org/llm-verifier/sdk/go"
)

func main() {
    client := verifier.NewClient("http://localhost:8080", "your-api-key")

    result, err := client.VerifyModel(context.Background(),
        verifier.VerificationRequest{
            ModelID:  "gpt-4o",
            Provider: "openai",
            APIKey:   "sk-your-key",
        })

    if err != nil {
        log.Fatal(err)
    }

    log.Printf("Model score: %.1f/100",
        result.PerformanceScores.OverallScore)
}
```

Python SDK

```
from llm_verifier import Client

client = Client("http://localhost:8080", "your-api-key")

result = client.verify_model(
    model_id="gpt-4o",
    provider="openai",
    api_key="sk-your-key"
)

print(f"Model score: {result.performance_scores.overall_score}/100")
```

JavaScript SDK

```
const { Client } = require('llm-verifier-sdk');
```

```
const client = new Client('http://localhost:8080', 'your-api-key');

client.verifyModel({
  model_id: 'gpt-4o',
  provider: 'openai',
  api_key: 'sk-your-key'
}).then(result => {
  console.log(`Model score: ${
    result.performance_scores.overall_score}/100`);
}).catch(console.error);
```

CLI Reference

Global Options

```
llm-verifier [command] [options]
```

Options:

<code>-h, --help</code>	Show help
<code>-v, --verbose</code>	Verbose output
<code>--debug</code>	Enable debug mode
<code>--config string</code>	Configuration file path
<code>--timeout duration</code>	Request timeout (default 30s)

Commands

verify

```
llm-verifier verify [models...] [flags]
```

Flags:

<code>--provider string</code>	AI provider (openai, anthropic, etc.)
<code>--model string</code>	Model ID to verify
<code>--parallel int</code>	Number of parallel verifications (default 3)
<code>--timeout duration</code>	Verification timeout (default 30s)
<code>--output string</code>	Output file for results
<code>--format string</code>	Output format (json, yaml, table)

export-config

```
llm-verifier export-config <format> [flags]
```

Formats: opencode, crush, claude-code

Flags:

<code>--output string</code>	Output file path
<code>--include-api-keys</code>	Include API keys in export

<code>--min-score float</code>	Minimum model score threshold
<code>--providers strings</code>	Specific providers to include

analytics

`llm-verifier analytics [flags]`

Flags:

<code>--since duration</code>	Show analytics since duration
<code>--provider string</code>	Filter by provider
<code>--format string</code>	Output format (table, json)

migrate-config

`llm-verifier migrate-config [flags]`

Flags:

<code>--input string</code>	Input config file
<code>--output string</code>	Output config file
<code>--backup</code>	Create backup of original

Configuration Files

YAML Configuration

```
# llm-verifier.yaml
api:
  port: 8080
  timeout: 30s
  rate_limit: 60

database:
  url: "postgres://user:pass@localhost/llm_verifier"
  max_connections: 10

providers:
  openai:
    api_key: "${OPENAI_API_KEY}"
    timeout: 30s
    retry_count: 3

  anthropic:
    api_key: "${ANTHROPIC_API_KEY}"
    timeout: 45s

verification:
  parallel_requests: 5
  default_timeout: 30s
  test_cases:
    - name: "code_generation"
```

```
    enabled: true
    timeout: 60s
  - name: "reasoning"
    enabled: true
    timeout: 45s

logging:
  level: "info"
  format: "json"
  output: "stdout"
```

Environment Variables

```
# API Configuration
LLM_VERIFIER_API_PORT=8080
LLM_VERIFIER_API_TIMEOUT=30s

# Database
LLM_VERIFIER_DATABASE_URL=postgres://...

# Provider API Keys
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=...
GROQ_API_KEY=...

# Logging
LLM_VERIFIER_LOG_LEVEL=info
LLM_VERIFIER_DEBUG=true
```

Monitoring & Metrics

Prometheus Metrics

```
# Request metrics
llm_verifier_requests_total{endpoint="/api/v1/verify",
method="POST"} 12543

# Response time percentiles
llm_verifier_duration_seconds{quantile="0.5", provider="openai"}
2.3
llm_verifier_duration_seconds{quantile="0.95", provider="openai"}
8.7

# Error rates
llm_verifier_errors_total{provider="openai", error_type="timeout"}
23

# Model scores
llm_verifier_model_score{model="gpt-4o", metric="overall"} 95.2
```

Health Check Endpoint

GET /health

Response:

```
{
  "status": "healthy",
  "timestamp": "2024-01-01T12:00:00Z",
  "version": "2.0.0",
  "uptime": "24h30m45s"
}
```

Troubleshooting API Issues

Common Error Responses

Invalid API Key

```
{
  "error": "PROVIDER_API_ERROR",
  "message": "Invalid API key provided",
  "code": 401
}
```

Rate Limit Exceeded

```
{
  "error": "PROVIDER_RATE_LIMIT",
  "message": "Rate limit exceeded. Try again later.",
  "code": 429,
  "retry_after": 60
}
```

Model Not Found

```
{
  "error": "MODEL_NOT_FOUND",
  "message": "Model 'gpt-5' not found for provider 'openai'",
  "code": 404
}
```

Debug Mode

Enable detailed API logging:

```
export LLM_VERIFIER_DEBUG=true
export LLM_VERIFIER_LOG_LEVEL=debug
```

This will log all API requests, responses, and timing information.

Version History

v2.0.0 (Current)

-  ProviderInitError resolution for OpenCode
-  Enhanced provider detection (20+ providers)
-  Intelligent model selection and agent assignment
-  Configuration migration tools
-  Analytics and monitoring system
-  Comprehensive REST API
-  Multiple export formats

v1.5.0

- Basic model verification
- Provider support (OpenAI, Anthropic)
- JSON export functionality
- Basic CLI interface

v1.0.0

- Initial release
- Core verification engine
- Basic provider support